SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE	DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Program Name: B. Tech	Assignment Type: Lab	Academic Year:2025-2026

Course Coordinator Name	Dr. Rishabh Mittal
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju

CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week8 – Friday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches

Assignment Number: 15.5(Present assignment number)/**24**(Total number of assignments)

Q.No.	Question	*Expected Time to complete*
1	**Lab 15 – Backend API Development: Creating RESTful services with AI** **Lab Objectives:** • Understand the fundamentals of RESTful API design. • Use AI-assisted coding tools to generate backend services. • Implement CRUD (Create, Read, Update, Delete) operations. • Document endpoints with comments and auto-generated documentation.	Week8 -
	Task 1 – Student Attendance API **Task:** **Create a RESTful API for student attendance tracking.**	

Instructions:

• Endpoints required:

o GET /attendance → View attendance records

o POST /attendance → Mark attendance

o PUT /attendance/{id} → Update attendance record

o DELETE /attendance/{id} → Remove attendance entry

• Ensure JSON-based request and response handling.

Expected Output:

• API for maintaining and updating attendance records.

Task 2 – Inventory Management API

Task:

Develop a REST API for warehouse inventory management.

Instructions:

• Endpoints required:

o GET /inventory → List all items

o POST /inventory → Add new item

o PUT /inventory/{id} → Update stock quantity

o DELETE /inventory/{id} → Remove an item

• Implement validation for stock values.

Expected Output:

• API capable of managing inventory stock levels.

Task 3 – Hotel Room Booking API

Task:

Develop a RESTful API for hotel room booking.

Instructions:

• Endpoints required:

o GET /rooms → View available rooms

o POST /rooms/book → Book a room

o GET /rooms/{id} → View booking details

o DELETE /rooms/{id} → Cancel booking

• Implement error handling for unavailable rooms.

Expected Output:

• API simulating hotel room booking operations.

Task 4 – Online Voting API

Task:

Create a RESTful API for an online voting system.

Instructions:

• Endpoints required:

o GET /candidates → List candidates

o POST /vote → Cast a vote

o GET /results → View voting results

• Ensure one vote per user using in-memory validation.

Expected Output:

• API simulating a secure online voting process.

Task 5 – Teacher Management API

Task:

Develop a RESTful API to manage teacher records.

Instructions:

• Endpoints required:

o GET /teachers → List all teachers

o POST /teachers → Add a new teacher

o PUT /teachers/{id} → Update teacher details

o DELETE /teachers/{id} → Remove a teacher

• Ensure proper error handling and JSON responses.

Expected Output:

• Functional API for managing teacher information.

Task 1 : Student Attendance API

Prompt:

Create a RESTful API for student attendance tracking with CRUD endpoints using JSON request and response handling. Ensure the API can run in a cloud notebook environment and be accessed through a public URL.

Code:

View Attendance Records (GET)

```
requests.get("http://127.0.0.1:5000/attendance").json()
```

Mark Attendance (POST)

```
import requests

data = {
   "name": "Ravi",
   "status": "Present"
}

requests.post("http://127.0.0.1:5000/attendance", json=data).json()
```

Update Attendance (PUT)

```
update = {
   "status": "Absent"
}

requests.put("http://127.0.0.1:5000/attendance/1", json=update).json()
```

Delete Attendance (DELETE)

```
requests.delete("http://127.0.0.1:5000/attendance/1").json()
```

Sample Input/Output:

View Attendance Records (GET)

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 16:12:37] "GET /attendance HTTP/1.1" 200 -
[]
```

Mark Attendance (POST)

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 16:13:07] "POST /attendance HTTP/1.1" 200 -
{'message': 'Attendance Added',
 'record': {'id': 2, 'name': 'Ravi', 'status': 'Present'}}
```

Update Attendance (PUT)

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 16:14:11] "PUT /attendance/1 HTTP/1.1" 200 -
{'error': 'Record Not Found'}
```

Delete Attendance (DELETE)

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 16:14:47] "DELETE /attendance/1 HTTP/1.1" 200 -
{'error': 'Record Not Found'}
```

Explanation:

1. This program implements a RESTful API using the Flask framework.
2. The system maintains student attendance records in an in-memory list.
3. The GET endpoint retrieves all attendance entries in JSON format.
4. The POST endpoint allows users to add new attendance records.
5. The PUT endpoint updates attendance status using a unique ID.
6. The DELETE endpoint removes attendance entries from the dataset.
7. JSON request and response handling ensures structured communication.
8. The API demonstrates CRUD operations for attendance management.

Task 2: Inventory Management API

Prompt:

Develop a RESTful API to manage inventory items with endpoints to list, add, update stock quantity, and delete items. Ensure all inventory data is handled using JSON-based request and response communication.

Code:

```python
from flask import Flask, request, jsonify
from threading import Thread

app = Flask(__name__)

inventory = []
current_id = 1


# ✅ GET → List all items
@app.route('/inventory', methods=['GET'])
def get_items():
    return jsonify(inventory)


# ✅ POST → Add new item
@app.route('/inventory', methods=['POST'])
def add_item():
    global current_id
    data = request.get_json()

    item = {
        "id": current_id,
        "name": data["name"],
        "quantity": data["quantity"]
    }

    inventory.append(item)
    current_id += 1

    return jsonify({"message": "Item Added", "item": item})


# ✅ PUT → Update stock quantity
@app.route('/inventory/<int:id>', methods=['PUT'])
def update_item(id):
    data = request.get_json()

    for item in inventory:
        if item["id"] == id:
            item["quantity"] = data["quantity"]
            return jsonify({"message": "Stock Updated", "item": item})

    return jsonify({"error": "Item Not Found"})


# ✅ DELETE → Remove item
@app.route('/inventory/<int:id>', methods=['DELETE'])
def delete_item(id):
    for item in inventory:
        if item["id"] == id:
            inventory.remove(item)
            return jsonify({"message": "Item Deleted"})

    return jsonify({"error": "Item Not Found"})


def run():
    app.run(port=5001)

Thread(target=run).start()
```

View Inventory

import requests

requests.get("http://127.0.0.1:5001/inventory").json()

Add new item

```
data = {
   "name": "Laptop",
   "quantity": 10
}

requests.post("http://127.0.0.1:5001/inventory", json=data).json()
```

Update stock quantity

```
update = {
   "quantity": 15
}

requests.put("http://127.0.0.1:5001/inventory/1", json=update).json()
```

Delete Item

```
requests.delete("http://127.0.0.1:5001/inventory/1").json()
```

Sample Input/Output

View Inventory

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 16:42:10] "GET /inventory HTTP/1.1" 200 -
[]
```

Add new item

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 16:42:22] "POST /inventory HTTP/1.1" 200 -
{'item': {'id': 1, 'name': 'Laptop', 'quantity': 10}, 'message': 'Item Added'}
```

Update Stock

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 16:43:15] "PUT /inventory/1 HTTP/1.1" 200 -
{'item': {'id': 1, 'name': 'Laptop', 'quantity': 15},
 'message': 'Stock Updated'}
```

Delete Item

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 16:43:49] "DELETE /inventory/1 HTTP/1.1" 200 -
{'message': 'Item Deleted'}
```

Explanation:

1. This program implements a RESTful API for inventory management using Flask.
2. The system stores item details such as item ID, name, and stock quantity.
3. The GET endpoint retrieves all available inventory items.
4. The POST endpoint allows users to add new items to inventory.
5. The PUT endpoint updates stock quantity using a unique item ID.
6. The DELETE endpoint removes items from inventory records.
7. JSON format ensures structured communication between client and server.
8. The API demonstrates full CRUD functionality for inventory control.

Task 3: Hotel Room Booking API

Prompt:

Develop a RESTful API to manage hotel room bookings with endpoints to view available rooms, book a room, view booking details, and cancel bookings.Ensure all interactions between client and server use JSON-formatted request and response data.

Code:

```python
from flask import Flask, request, jsonify
from threading import Thread

app = Flask(__name__)

rooms = [
    {"id": 1, "type": "Single", "booked": False},
    {"id": 2, "type": "Double", "booked": False},
    {"id": 3, "type": "Suite", "booked": False}
]

bookings = {}


# ✅ GET → View available rooms
@app.route('/rooms', methods=['GET'])
def view_rooms():
    available = [room for room in rooms if not room["booked"]]
    return jsonify(available)


# ✅ POST → Book a room
@app.route('/rooms/book', methods=['POST'])
def book_room():
    data = request.get_json()
    room_id = data["room_id"]
    customer = data["customer"]

    for room in rooms:
        if room["id"] == room_id and not room["booked"]:
            room["booked"] = True
            bookings[room_id] = {
                "room_id": room_id,
                "customer": customer
            }
            return jsonify({"message": "Room Booked", "booking": bookings[room_id]})

    return jsonify({"error": "Room not available"})


# ✅ GET → View booking details
@app.route('/rooms/<int:id>', methods=['GET'])
def booking_details(id):
    if id in bookings:
        return jsonify(bookings[id])
    return jsonify({"error": "Booking not found"})


# ✅ DELETE → Cancel booking
@app.route('/rooms/<int:id>', methods=['DELETE'])
def cancel_booking(id):
    if id in bookings:
        bookings.pop(id)
        for room in rooms:
            if room["id"] == id:
                room["booked"] = False
        return jsonify({"message": "Booking Cancelled"})
    return jsonify({"error": "Booking not found"})


def run():
    app.run(port=5002)

Thread(target=run).start()
```

View Available Rooms

```
import requests
requests.get("http://127.0.0.1:5002/rooms").json()
```

Book Room

```
data = {
   "room_id": 1,
   "customer": "Bhagath"
}
```

View Booking Details

```
requests.get("http://127.0.0.1:5002/rooms/1").json()
```

Cancel Booking

```
requests.delete("http://127.0.0.1:5002/rooms/1").json()
```

Sample Input/Output:

View Available Rooms

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 17:02:30] "GET /rooms HTTP/1.1" 200 -
[{'booked': False, 'id': 1, 'type': 'Single'},
 {'booked': False, 'id': 2, 'type': 'Double'},
 {'booked': False, 'id': 3, 'type': 'Suite'}]
```

Book Room

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 17:03:23] "POST /rooms/book HTTP/1.1" 200 -
{'booking': {'customer': 'Bhagath', 'room_id': 1}, 'message': 'Room Booked'}
```

View Booking Details

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 17:04:06] "GET /rooms/1 HTTP/1.1" 200 -
{'customer': 'Bhagath', 'room_id': 1}
```

Cancel Booking

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 17:04:42] "DELETE /rooms/1 HTTP/1.1" 200 -
{'message': 'Booking Cancelled'}
```

Explanation:

1. This program implements a RESTful API for hotel room booking management.
2. The system maintains room availability and booking details in memory.
3. The GET endpoint lists all currently available rooms.
4. The POST endpoint allows customers to book a room.
5. The GET endpoint with room ID retrieves booking information.
6. The DELETE endpoint cancels an existing room booking.
7. JSON-based request and response ensure proper API communication.
8. The API demonstrates real-time booking control using CRUD operations.

Task 4: Online Voting API

Prompt:

Develop a RESTful API for an online voting system that allows users to view candidates, cast votes, and check voting results. Ensure all data communication between client and server is handled using JSON format.

Code:

```python
from flask import Flask, request, jsonify
from threading import Thread

app = Flask(__name__)

candidates = [
    {"id": 1, "name": "Alice", "votes": 0},
    {"id": 2, "name": "Bob", "votes": 0},
    {"id": 3, "name": "Charlie", "votes": 0}
]


# ✅ GET → List candidates
@app.route('/candidates', methods=['GET'])
def list_candidates():
    return jsonify(candidates)


# ✅ POST → Cast vote
@app.route('/vote', methods=['POST'])
def cast_vote():
    data = request.get_json()
    cid = data["candidate_id"]

    for c in candidates:
        if c["id"] == cid:
            c["votes"] += 1
            return jsonify({"message": "Vote Casted", "candidate": c})

    return jsonify({"error": "Candidate not found"})


# ✅ GET → View results
@app.route('/results', methods=['GET'])
def results():
    return jsonify(candidates)


def run():
    app.run(port=5006)

Thread(target=run).start()
```

View Candidates

import requests

requests.get("http://127.0.0.1:5003/candidates").json()

Cast Vote

```
vote = {
   "candidate_id": 1
}

requests.post("http://127.0.0.1:5003/vote", json=vote).json()
```

View Voting Results

```
requests.get("http://127.0.0.1:5003/results").json()
```

Sample Input/Output:

```
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5004
```

View Candidates

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 17:21:06] "GET /candidates HTTP/1.1" 200 -
[{'id': 1, 'name': 'Alice', 'votes': 0},
 {'id': 2, 'name': 'Bob', 'votes': 0},
 {'id': 3, 'name': 'Charlie', 'votes': 0}]
```

Cast Vote

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 17:21:48] "POST /vote HTTP/1.1" 200 -
{'candidate': {'id': 1, 'name': 'Alice', 'votes': 1}, 'message': 'Vote Casted'}
```

View Voting Results

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 17:23:19] "GET /results HTTP/1.1" 200 -
[{'id': 1, 'name': 'Alice', 'votes': 1},
 {'id': 2, 'name': 'Bob', 'votes': 0},
 {'id': 3, 'name': 'Charlie', 'votes': 0}]
```

Explanation:

1. This program implements a RESTful API for an online voting system.
2. The system stores candidate details and vote counts in memory.
3. The GET endpoint lists all available candidates.
4. The POST endpoint allows users to cast votes securely.
5. Each vote increases the respective candidate’s vote count.
6. The GET results endpoint displays updated voting statistics.
7. JSON communication ensures structured client-server interaction.
8. The API demonstrates real-time vote management functionality.

Task 5 : Teacher Management API

Prompt:

Develop a RESTful API to manage teacher records with endpoints to list teachers, add new teachers, update teacher details, and delete teacher entries. Ensure all client-server communication is handled using JSON-formatted requests and responses.

Code:

```python
from flask import Flask, request, jsonify
from threading import Thread

app = Flask(__name__)

teachers = []
current_id = 1


# ✅ GET → List all teachers
@app.route('/teachers', methods=['GET'])
def get_teachers():
    return jsonify(teachers)


# ✅ POST → Add new teacher
@app.route('/teachers', methods=['POST'])
def add_teacher():
    global current_id
    data = request.get_json()

    teacher = {
        "id": current_id,
        "name": data["name"],
        "subject": data["subject"]
    }

    teachers.append(teacher)
    current_id += 1

    return jsonify({"message": "Teacher Added", "teacher": teacher})


# ✅ PUT → Update teacher details
@app.route('/teachers/<int:id>', methods=['PUT'])
def update_teacher(id):
    data = request.get_json()

    for t in teachers:
        if t["id"] == id:
            t["name"] = data.get("name", t["name"])
            t["subject"] = data.get("subject", t["subject"])
            return jsonify({"message": "Teacher Updated", "teacher": t})

    return jsonify({"error": "Teacher Not Found"})


# ✅ DELETE → Remove teacher
@app.route('/teachers/<int:id>', methods=['DELETE'])
def delete_teacher(id):
    for t in teachers:
        if t["id"] == id:
            teachers.remove(t)
            return jsonify({"message": "Teacher Deleted"})

    return jsonify({"error": "Teacher Not Found"})


def run():
    app.run(port=5000)

Thread(target=run).start()
```

View Teachers

```
import requests
requests.get("http://127.0.0.1:5000/teachers").json()
```

Add Teacher

```
data = {
   "name": "Dr. Rao",
   "subject": "Machine Learning"
}

requests.post("http://127.0.0.1:5000/teachers", json=data).json()
```

Verify

```
requests.get("http://127.0.0.1:5000/teachers").json()
```

Update Teacher

```
update = {
   "subject": "Artificial Intelligence"
}

requests.put("http://127.0.0.1:5000/teachers/1", json=update).json()
```

Delete Teacher

```
requests.delete("http://127.0.0.1:5000/teachers/1").json()
```

Sample Input/Output:

```
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
```

View Teachers

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 17:39:47] "GET /teachers HTTP/1.1" 200 -
[]
```

Add Teacher

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 17:40:55] "POST /teachers HTTP/1.1" 200 -
{'message': 'Teacher Added',
 'teacher': {'id': 1, 'name': 'Dr. Rao', 'subject': 'Machine Learning'}}
```

Verify

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 17:41:48] "GET /teachers HTTP/1.1" 200 -
[{'id': 1, 'name': 'Dr. Rao', 'subject': 'Machine Learning'}]
```

Update Teacher

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 17:42:35] "PUT /teachers/1 HTTP/1.1" 200 -
{'message': 'Teacher Updated',
 'teacher': {'id': 1, 'name': 'Dr. Rao', 'subject': 'Artificial Intelligence'}}
```

Delete Teacher

```
INFO:werkzeug:127.0.0.1 - - [13/Mar/2026 17:43:12] "DELETE /teachers/1 HTTP/1.1" 200 -
{'message': 'Teacher Deleted'}
```

Explanation:

1. This program implements a RESTful API for teacher record management using Flask.
2. The system maintains teacher details such as ID, name, and subject.
3. The GET endpoint retrieves all teacher records.
4. The POST endpoint allows adding new teachers to the system.
5. The PUT endpoint updates teacher information using a unique ID.
6. The DELETE endpoint removes teacher records from the dataset.
7. JSON-based request and response handling ensures structured API communication.
8. The API demonstrates complete CRUD functionality for teacher management.